

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №6
з дисципліни
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44
Крошка Дар'я Олександрівна
номер у списку групи: 34

Перевірила:

Молчанова А. А.

Київ 2024

Завдання:

1. Задано двовимірний масив (матрицю) цілих чисел $A[m,n]$ або $A[n,n]$, де m та n – натуральні числа (константи), що визначають розміри двовимірного масиву. Виконати сортування цього масиву або заданої за варіантом його частини у заданому порядку заданим алгоритмом (методом).

Сортування повинно бути виконано безпосередньо у двовимірному масиві «на тому ж місці», тобто без перезаписування масиву та/або його будь-якої частини до інших одно- або двовимірних масивів, а також без використання спискових структур даних.

2. Розміри матриці m та n взяти самостійно у межах від 7 до 10.

3. При тестуванні програми необхідно підбирати такі вхідні набори початкових значень матриці, щоб можна було легко відстежити коректність виконання сортування і ця коректність була б

протестована для всіх можливих випадків. З метою тестування дозволяється використовувати матриці меншого розміру.

Варіант:

Варіант № 4

Задано квадратну двовимірний масив (матрицю) цілих чисел $A[n,n]$. Відсортувати головну діагональ масиву алгоритмом №4 методу вставки (з двійковим пошуком) за незбільшенням.

Текст програми:

```
#include <stdio.h>
```

```
void binaryInsertionSortDescending(float arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        float t = arr[i];
```

```

    int L = 0, R = i;

    while (L < R) {
        int j = (L + R) / 2;
        if (arr[j] >= T)
            L = j + 1;
        else
            R = j;
    }

    for (int k = i; k > R; k--)
        arr[k] = arr[k - 1];

    arr[R] = T;
}

}

void sortMainDiagonal(float matrix[8][8]) {
    float diagonal[8];

    for (int i = 0; i < 8; i++)
        diagonal[i] = matrix[i][i];

    binaryInsertionSortDescending(diagonal, 8);

    for (int i = 0; i < 8; i++)
        matrix[i][i] = diagonal[i];
}

int main() {
    float matrix[8][8] = {
        {45, 20, 13, 44, 58, 16, 72, 18},
        {11, 38, 59, 10, 11, 12, 13, 14},
        {15, 16, 29, 18, 19, 20, 21, 22},
        {23, 24, 25, 12, 27, 28, 29, 30},
        {31, 32, 33, 34, 56, 36, 37, 38},
        {39, 40, 41, 42, 43, 17, 45, 46},

```

```

        {47, 48, 49, 50, 51, 52, 22, 54},
        {55, 56, 57, 58, 59, 60, 61, 13}
};

printf("Original matrix:\n");
for (int i = 0; i < 8; i++) {
    for (int j = 0; j < 8; j++) {
        printf("%.2f ", matrix[i][j]);
    }
    printf("\n");
}

sortMainDiagonal(matrix);

printf("\nMatrix after sorting the main diagonal in descending
order:\n");
for (int i = 0; i < 8; i++) {
    for (int j = 0; j < 8; j++) {
        printf("%.2f ", matrix[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Результати тестування (для різних матриць):

```
Original matrix:
45.00 20.00 13.00 44.00 58.00 16.00 72.00 18.00
11.00 38.00 59.00 10.00 11.00 12.00 13.00 14.00
15.00 16.00 29.00 18.00 19.00 20.00 21.00 22.00
23.00 24.00 25.00 12.00 27.00 28.00 29.00 30.00
31.00 32.00 33.00 34.00 56.00 36.00 37.00 38.00
39.00 40.00 41.00 42.00 43.00 17.00 45.00 46.00
47.00 48.00 49.00 50.00 51.00 52.00 22.00 54.00
55.00 56.00 57.00 58.00 59.00 60.00 61.00 13.00

Matrix after sorting the main diagonal in descending order:
56.00 20.00 13.00 44.00 58.00 16.00 72.00 18.00
11.00 45.00 59.00 10.00 11.00 12.00 13.00 14.00
15.00 16.00 38.00 18.00 19.00 20.00 21.00 22.00
23.00 24.00 25.00 29.00 27.00 28.00 29.00 30.00
31.00 32.00 33.00 34.00 22.00 36.00 37.00 38.00
39.00 40.00 41.00 42.00 43.00 17.00 45.00 46.00
47.00 48.00 49.00 50.00 51.00 52.00 13.00 54.00
55.00 56.00 57.00 58.00 59.00 60.00 61.00 12.00
```

```
Original matrix:
99.00 1.00 2.00 3.00 4.00 5.00 6.00 7.00
8.00 88.00 9.00 10.00 11.00 12.00 13.00 14.00
15.00 16.00 77.00 17.00 18.00 19.00 20.00 21.00
22.00 23.00 24.00 66.00 25.00 26.00 27.00 28.00
29.00 30.00 31.00 32.00 55.00 33.00 34.00 35.00
36.00 37.00 38.00 39.00 40.00 44.00 41.00 42.00
43.00 45.00 46.00 47.00 48.00 49.00 33.00 50.00
51.00 52.00 53.00 54.00 55.00 56.00 57.00 22.00

Matrix after sorting the main diagonal in descending order:
99.00 1.00 2.00 3.00 4.00 5.00 6.00 7.00
8.00 88.00 9.00 10.00 11.00 12.00 13.00 14.00
15.00 16.00 77.00 17.00 18.00 19.00 20.00 21.00
22.00 23.00 24.00 66.00 25.00 26.00 27.00 28.00
29.00 30.00 31.00 32.00 55.00 33.00 34.00 35.00
36.00 37.00 38.00 39.00 40.00 44.00 41.00 42.00
43.00 45.00 46.00 47.00 48.00 49.00 33.00 50.00
51.00 52.00 53.00 54.00 55.00 56.00 57.00 22.00
```

```
Original matrix:
-12.00 45.00 23.00 -56.00 78.00 -89.00 12.00 -34.00
54.00 -11.00 67.00 22.00 -90.00 31.00 45.00 -76.00
-87.00 32.00 -19.00 24.00 68.00 -12.00 56.00 10.00
11.00 -78.00 34.00 15.00 -89.00 45.00 -90.00 62.00
90.00 -45.00 31.00 -68.00 13.00 72.00 -81.00 50.00
-56.00 12.00 -89.00 43.00 -20.00 17.00 33.00 71.00
78.00 -56.00 90.00 11.00 -12.00 45.00 -22.00 63.00
10.00 -89.00 34.00 -90.00 45.00 -61.00 78.00 -99.00

Matrix after sorting the main diagonal in descending order:
17.00 45.00 23.00 -56.00 78.00 -89.00 12.00 -34.00
54.00 15.00 67.00 22.00 -90.00 31.00 45.00 -76.00
-87.00 32.00 13.00 24.00 68.00 -12.00 56.00 10.00
11.00 -78.00 34.00 -11.00 -89.00 45.00 -90.00 62.00
90.00 -45.00 31.00 -68.00 -12.00 72.00 -81.00 50.00
-56.00 12.00 -89.00 43.00 -20.00 -19.00 33.00 71.00
78.00 -56.00 90.00 11.00 -12.00 45.00 -22.00 63.00
10.00 -89.00 34.00 -90.00 45.00 -61.00 78.00 -99.00
```

```
Original matrix:
0.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00
0.00 2.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 3.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 4.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 5.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 6.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 7.00 0.00
8.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00

Matrix after sorting the main diagonal in descending order:
7.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00
0.00 6.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 5.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 4.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 3.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 2.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
8.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
```

Висновок:

У ході виконання даної роботи мною було реалізовано алгоритм сортування головної діагоналі квадратної матриці методом прямого вставлення із застосуванням двійкового пошуку.

Написана програма:

- Виконує сортування елементів виключно на головній діагоналі матриці.
- Забезпечує зміну позицій елементів без використання додаткових структур даних.

У ході тестування програма продемонструвала правильну роботу в усіх випадках:

- для вже відсортованої діагоналі;
- для діагоналі з випадковими значеннями;
- для діагоналі, відсортованої у зворотному порядку;